

Web-Only Ultra-Lightweight Music Player: Architecture & Design Document

This document outlines the comprehensive architectural design, technology stack, and implementation strategy for a high-performance, web-only music player. The system is engineered to operate efficiently across a highly diverse range of hardware, spanning from low-end mobile devices and legacy desktop browsers to modern Smart TVs. The primary architectural mandate is to maintain a "near-instant" operational latency and continuous background playback.

1. System Architecture Overview

The platform utilizes a hybrid deployment architecture, decoupling the client-side application from the media storage and streaming backend to completely eliminate egress costs while maximizing global delivery speeds.

- **Code Repository & Versioning:** Managed via GitHub to facilitate CI/CD pipelines.
- **Frontend Hosting (The Thin Client):** Cloudflare Pages provides edge-hosted distribution for the static web application.
- **Media Storage (The Vault):** Cloudflare R2 serves as the primary object store for audio files, offering zero-egress bandwidth costs.
- **Edge API & Streaming (The Proxy):** Cloudflare Workers mediate all requests between the client and R2, handling partial content range requests and authentication.
- **Database (The Index):** Cloudflare D1 (SQL at the Edge) stores lightweight metadata to prevent high-latency bucket scans.

2. Technology Stack

Component	Technology Selection	Strategic Rationale
Frontend Framework	Preact & Vite	Minimal 3kB payload. Removes virtual DOM overhead for CPU-constrained TV

Component	Technology Selection	Strategic Rationale
		environments.
Styling Engine	Tailwind CSS v4	Zero-runtime CSS parsing; crucial for avoiding UI blockages during render.
Database	Cloudflare D1	Serverless SQLite database localized to the edge for sub-10ms metadata retrieval.
Audio Playback	HTML5 Audio & Media Session API	Native OS integration for lock-screen controls and background persistence on mobile.

3. Monorepo File Structure

The project will utilize a lightweight monorepo structure to keep frontend logic and Edge API logic synchronized.

```
/root
├── .github/workflows/    # CI/CD automation pipelines
├── /apps
│   ├── /web-player      # Client Application
│   │   ├── /public      # PWA Manifest, Service Workers, Icons
│   │   ├── /src
│   │   │   ├── /components  # UI Elements (Player, TrackList)
│   │   │   ├── /hooks      # Custom logic (useAudio, useSpatialNav)
│   │   │   ├── /services   # Cloudflare Worker API interfaces
│   │   │   └── main.tsx    # Application entry point
```

```
|   |— tailwind.config.js
|   |— vite.config.ts
|— /services
|   |— /edge-api      # Cloudflare Worker Backend
|   |— src/index.ts   # Route handlers (Streaming, Uploads, Meta)
|   |— wrangler.toml  # R2 bucket and D1 database bindings
|— schema.sql         # Database migration definitions
```

4. Core Implementation Strategies

4.1 Continuous Playback & Memory Management

To ensure music does not stop when mobile browsers background the tab, the application will leverage the **Media Session API**. Furthermore, to provide gapless playback and prevent Out-Of-Memory (OOM) crashes on low-end devices, a **Double-Buffering** approach will be utilized. The system will pre-load the first 10 seconds of the sequential track into a hidden audio element while actively purging previous audio blobs via `URL.revokeObjectURL()`.

4.2 TV-First Spatial Navigation

Smart TVs operate without pointer devices. The UI will implement a `useSpatialNav` hook to capture directional `keydown` events (ArrowUp, ArrowDown, Enter). Active focus will be visually highlighted using high-contrast CSS outlines, and `scrollIntoView({ behavior: "smooth", block: "center" })` will ensure the focused track remains centered on the display.

4.3 Edge-Proxied Range Streaming

To support track seeking and minimize data consumption, the Cloudflare Worker will intercept standard HTTP requests and translate them into R2 Range requests. This ensures the browser only downloads the exact bytes required for the current playback position rather than buffering the entire file.

5. Database Schema (Cloudflare D1)

The relational database is kept intentionally flat to prioritize read-speed for library loading.

Field Name	Data Type	Description
id	TEXT (UUID)	Primary Key.
title	TEXT	Track name extracted during Edge upload.
artist	TEXT	Track artist extracted during Edge upload.
duration	INTEGER	Length of track in seconds (used for UI rendering without file load).
r2_key	TEXT	The exact storage path within the Cloudflare R2 bucket.

6. Deployment Pipeline

1. **Code Commit:** Developer pushes updates to the `main` branch of the GitHub repository.
2. **GitHub Actions Trigger:** A workflow intercepts the push and initiates the build process.
3. **Frontend Deployment:** The static Preact/Vite build is dispatched to Cloudflare Pages.
4. **Backend Synchronization:** Changes to the Worker script or `wrangler.toml` are deployed to the Cloudflare Edge network, instantly updating the API globally.